

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge
Representation Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED
COURSE CODE: MLA01

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.* (BTL 3) Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 60 Mins No. of Questions: 5
Annexure A

Knowledge Representation Design Assessment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	

6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	9%	
10.Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

1. Representation Using Propositional Logic

In **Propositional Logic**, each statement is represented by a proposition that can be either **True or False**.

Step 1: Define Propositions

Let:

- **F** = Ravi has fever.
- **C** = Ravi has cough.
- **B** = Ravi has body pain.
- **L** = Ravi has flu.
- **V** = Ravi has viral infection.
- **R** = Ravi should be advised to rest.

Step 2: Represent the Facts

The given facts are:

1. Ravi has fever.

F

2. Ravi has cough.

C

3. Ravi has body pain.

B

Step 3: Represent the Rules

Rule 1: Patients with fever and cough may have flu.

$$(F \wedge C) \rightarrow L$$

Rule 2: Patients with flu and body pain may have viral infection.

$$(L \wedge B) \rightarrow V$$

Rule 3: Patients with viral infection should be advised to rest.

$$V \rightarrow R$$

Complete Propositional Knowledge Base

$$KB = \{F, C, B, (F \wedge C) \rightarrow L, (L \wedge B) \rightarrow V, V \rightarrow R\}$$

Inference

From:

$$F \wedge C$$

we infer:

$$L$$

Then:

$$L \wedge B$$

gives:

V

Finally:

V

gives:

R

Therefore:

R

Conclusion: Ravi should be advised to rest.

2. Conversion of Knowledge into First-Order Logic (FOL)

In **First-Order Logic**, we represent the knowledge using **constants, predicates, variables, and quantifiers**.

Step 1: Define the Constant

- **Ravi** = the patient Ravi.

Step 2: Define Predicates

Let:

- $\text{Fever}(x)$ = x has fever
- $\text{Cough}(x)$ = x has cough
- $\text{BodyPain}(x)$ = x has body pain
- $\text{Flu}(x)$ = x has flu

- $\text{ViralInfection}(x) = x$ has viral infection
- $\text{Rest}(x) = x$ should be advised to rest

Here, x represents any patient.

Step 3: Represent the Facts

Fact 1: Ravi has fever

$\text{Fever}(\text{Ravi})$

Fact 2: Ravi has cough

$\text{Cough}(\text{Ravi})$

Fact 3: Ravi has body pain

$\text{BodyPain}(\text{Ravi})$

Step 4: Represent the Rules

Rule 1: Patients with fever and cough may have flu

$\forall x[(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)]$

Rule 2: Patients with flu and body pain may have viral infection

$\forall x[(\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x)]$

Rule 3: Patients with viral infection should be advised to rest

$\forall x[\text{ViralInfection}(x) \rightarrow \text{Rest}(x)]$

Complete FOL Knowledge Base

$\text{Fever}(\text{Ravi}) \wedge \text{Cough}(\text{Ravi}) \wedge \text{BodyPain}(\text{Ravi}) \wedge \forall x[(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)] \wedge \forall x[(\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x)] \wedge \forall x[\text{ViralInfection}(x) \rightarrow \text{Rest}(x)]$

Inference

For $x = \text{Ravi}$:

$\text{Fever}(\text{Ravi}) \wedge \text{Cough}(\text{Ravi}) \rightarrow \text{Flu}(\text{Ravi})$

Then:

$\text{Flu}(\text{Ravi}) \wedge \text{BodyPain}(\text{Ravi}) \rightarrow \text{ViralInfection}(\text{Ravi})$

Finally:

$\text{ViralInfection}(\text{Ravi}) \rightarrow \text{Rest}(\text{Ravi})$

Final Conclusion

$\text{Rest}(\text{Ravi})$

Therefore, according to the given knowledge base, **Ravi should be advised to rest.**

3. Construct a Knowledge Base Containing Facts and Rules

A **Knowledge Base (KB)** contains two main components:

1. **Facts** – Information that is known to be true.

2. **Rules** – Conditions used to derive new facts from existing facts.

A. Facts

The following facts are given about Ravi:

Fever(Ravi)

Cough(Ravi)

BodyPain(Ravi)

These mean:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.

B. Rules

The following rules are used for reasoning:

Rule 1:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

If a patient has fever and cough, then the patient may have flu.

Rule 2:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

If a patient has flu and body pain, then the patient may have a viral infection.

Rule 3:

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

If a patient has a viral infection, then the patient should be advised to rest.

C. Complete Knowledge Base

KNOWLEDGE BASE

FACTS:

1. Fever(Ravi)
2. Cough(Ravi)
3. BodyPain(Ravi)

RULES:

1. $\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
2. $\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$
3. $\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

D. Knowledge Base Representation

$\text{KB} = \text{Facts} + \text{Rules}$

Therefore,

$\text{KB} = \{\text{Fever}(\text{Ravi}), \text{Cough}(\text{Ravi}), \text{BodyPain}(\text{Ravi}), \text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$
 $\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x) \text{ ViralInfection}(x) \rightarrow \text{Rest}(x)\}$

E. Derived Knowledge

Using the facts and rules:

Fever(Ravi) + Cough(Ravi)

↓

Flu(Ravi)

↓

Flu(Ravi) + BodyPain(Ravi)

↓

ViralInfection(Ravi)

↓

Rest(Ravi)

Final inference: Rest(Ravi) — Ravi should be advised to rest.

4. Demonstrate Unification for at Least Two Predicates

Definition

Unification is the process of finding a suitable substitution for variables so that two predicates become identical.

The substitution that makes the predicates identical is called the **Unifier**. The most general substitution is called the **Most General Unifier (MGU)**.

Unification 1: Fever(x) and Fever(Ravi)

Consider the two predicates:

Fever(x)

and

$\text{Fever}(\text{Ravi})$

To make both predicates identical, substitute:

$x = \text{Ravi}$

Therefore, the substitution is:

$\{x/\text{Ravi}\}$

After substitution:

$\text{Fever}(\text{Ravi}) = \text{Fever}(\text{Ravi})$

Hence, the predicates successfully unify.

MGU:

$\theta_1 = \{x/\text{Ravi}\}$

Unification 2: $\text{Flu}(x)$ and $\text{Flu}(\text{Ravi})$

Consider:

$\text{Flu}(x)$

and

$\text{Flu}(\text{Ravi})$

Substitute:

$x = \text{Ravi}$

Therefore:

$\{x/\text{Ravi}\}$

After substitution:

$\text{Flu}(\text{Ravi}) = \text{Flu}(\text{Ravi})$

Hence, the predicates successfully unify.

MGU:

$\theta_2 = \{x/\text{Ravi}\}$

Summary

Predicate 1 Predicate 2 Substitution Result

Fever(x)	Fever(Ravi)	$\{x/\text{Ravi}\}$	Fever(Ravi)
----------	-------------	---------------------	-------------

Flu(x)	Flu(Ravi)	$\{x/\text{Ravi}\}$	Flu(Ravi)
--------	-----------	---------------------	-----------

Conclusion

Unification allows the general variable **x** in the knowledge-base rules to be replaced with the specific constant **Ravi**. Thus, the general rules can be applied specifically to Ravi.

$\text{MGU} = \{x/\text{Ravi}\}$

5. Forward Chaining to Determine Ravi's Condition

Definition

Forward chaining is a **data-driven inference technique**. It starts with the known facts and applies the rules step-by-step to derive new facts until the required conclusion is obtained.

Initial Facts

The knowledge base contains:

Fever(Ravi) Cough(Ravi) BodyPain(Ravi)

Rules

Rule 1:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

Rule 2:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

Rule 3:

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

Step 1: Apply Rule 1

We know:

Fever(Ravi)

and

Cough(Ravi)

Both conditions of Rule 1 are satisfied.

Therefore:

Flu(Ravi)

New fact: Ravi has flu.

Step 2: Apply Rule 2

Now we have:

Flu(Ravi)

and the initial fact:

BodyPain(Ravi)

Both conditions of Rule 2 are satisfied.

Therefore:

ViralInfection(Ravi)

New fact: Ravi may have a viral infection.

Step 3: Apply Rule 3

We now have:

ViralInfection(Ravi)

Therefore, Rule 3 gives:

Rest(Ravi)

New fact: Ravi should be advised to rest.

Forward Chaining Sequence

Initial Facts

|
|— Fever(Ravi)
|— Cough(Ravi)
└— BodyPain(Ravi)

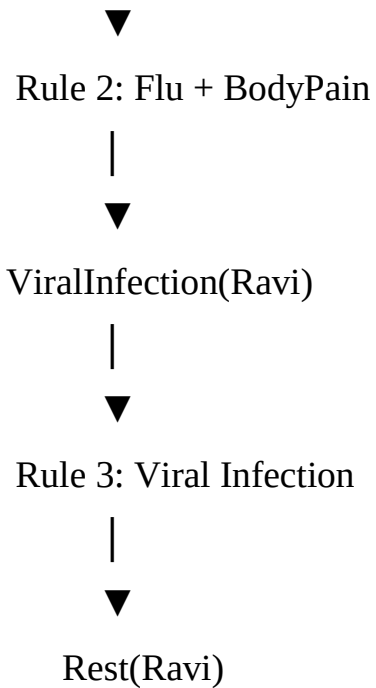
|
▼

Rule 1: Fever + Cough

|
▼

Flu(Ravi)

|



Final Inference

$\text{Fever(Ravi)} + \text{Cough(Ravi)} \rightarrow \text{Flu(Ravi)}$

$\text{Flu(Ravi)} + \text{BodyPain(Ravi)} \rightarrow \text{ViralInfection(Ravi)}$

$\text{ViralInfection(Ravi)} \rightarrow \text{Rest(Ravi)}$

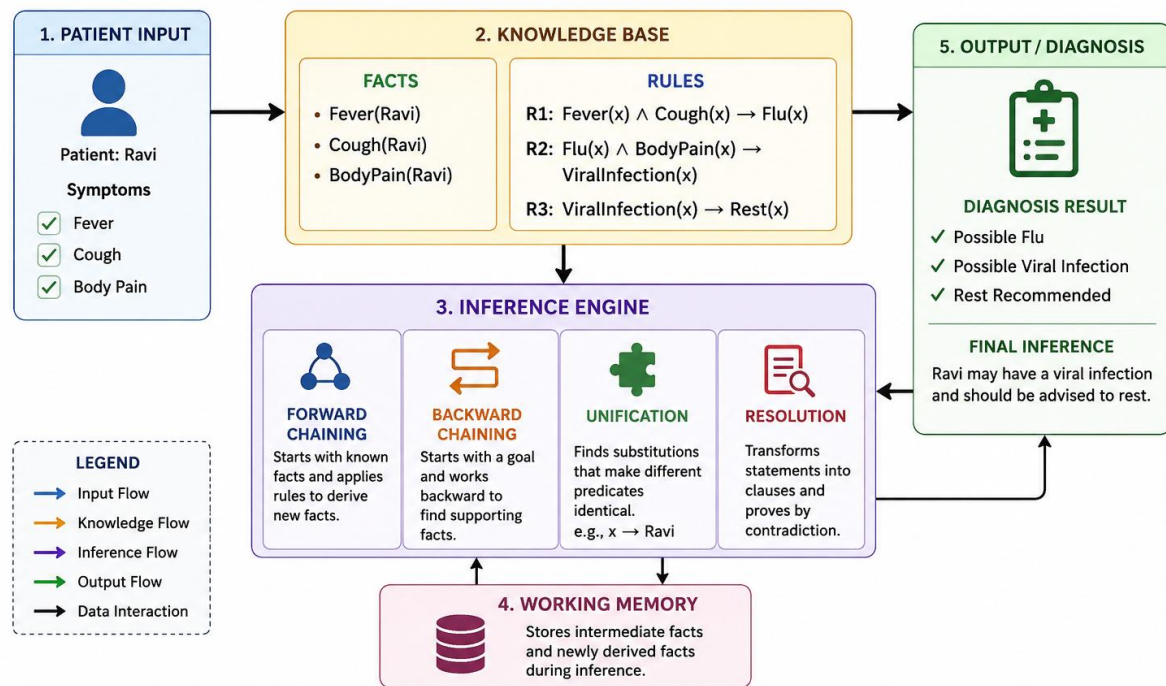
Conclusion

Using **forward chaining**, the system determines that:

ARCHITECTURE DIAGRAM:

SMART HEALTHCARE DIAGNOSIS SYSTEM

SYSTEM ARCHITECTURE DIAGRAM



Ravi has flu, which may lead to a viral infection, and he should be advised to rest.

6. Use Backward Chaining to Prove Whether Ravi Needs Rest

Definition

Backward chaining is a **goal-driven inference technique**. It starts with the desired goal and works backward through the rules to find whether the required facts are available in the knowledge base.

Goal

We need to prove:

$\text{Rest}(\text{Ravi})$

Step 1: Start with the Goal

Our goal is:

$\text{Rest}(\text{Ravi})$

Look for a rule whose conclusion is $\text{Rest}(x)$.

The rule is:

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

Therefore, to prove:

$\text{Rest}(\text{Ravi})$

we must prove:

$\text{ViralInfection}(\text{Ravi})$

Step 2: Prove Viral Infection

Look for a rule that concludes $\text{ViralInfection}(x)$:

$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$

Therefore, we need to prove:

$\text{Flu}(\text{Ravi})$

and

$\text{BodyPain}(\text{Ravi})$

$\text{BodyPain}(\text{Ravi})$ is already a known fact.

Now we need to prove:

$\text{Flu}(\text{Ravi})$

Step 3: Prove Flu

Look for a rule that concludes $\text{Flu}(x)$:

$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$

Therefore, we need:

$\text{Fever}(\text{Ravi})$

and

$\text{Cough}(\text{Ravi})$

Both are already known facts.

Therefore:

Flu(Ravi)

Step 4: Work Forward to the Goal

Since we have:

Flu(Ravi)

and

BodyPain(Ravi)

we can derive:

ViralInfection(Ravi)

Then:

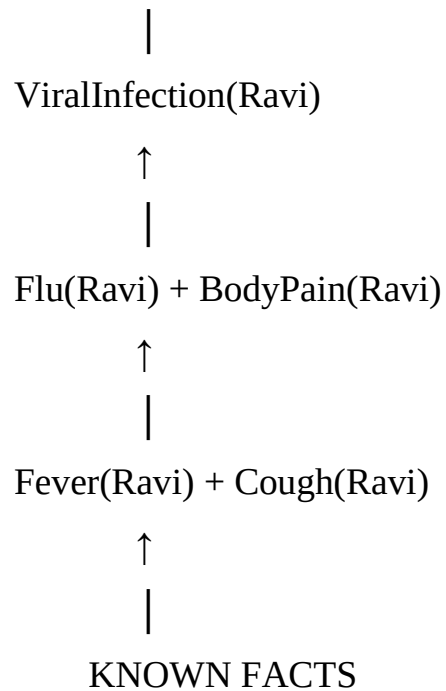
$\text{ViralInfection(Ravi)} \rightarrow \text{Rest(Ravi)}$

Therefore:

Rest(Ravi)

Backward Chaining Diagram

GOAL
Rest(Ravi)
↑



Proof

$\text{Rest(Ravi)} \Leftarrow \text{ViralInfection(Ravi)} \Leftarrow \text{Flu(Ravi)} \wedge \text{BodyPain(Ravi)}$
 $\Leftarrow \text{Fever(Ravi)} \wedge \text{Cough(Ravi)} \wedge \text{BodyPain(Ravi)}$

All three required facts are present in the knowledge base:

- Fever(Ravi) ✓
- Cough(Ravi) ✓
- BodyPain(Ravi) ✓

Conclusion

The backward-chaining process successfully proves:

Rest(Ravi)

Therefore, Ravi needs to be advised to rest.

7. Demonstrate Resolution for One Selected Query

Selected Query

We select the query:

Rest(Ravi)

We will use **resolution by refutation** to prove that Ravi should be advised to rest.

Step 1: Convert Rules into Clause Form

The rules are:

Rule 1:

$$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$$

Convert to clause form:

$$\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$$

Rule 2:

$$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$$

Clause form:

$$\neg \text{Flu}(x) \vee \neg \text{BodyPain}(x) \vee \text{ViralInfection}(x)$$

Rule 3:

$$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$$

Clause form:

$\neg \text{ViralInfection}(x) \vee \text{Rest}(x)$

Step 2: List the Known Facts

The knowledge base contains:

Fever(Ravi) Cough(Ravi) BodyPain(Ravi)

Step 3: Negate the Query

To prove Rest(Ravi) using resolution, assume the opposite:

$\neg \text{Rest}(\text{Ravi})$

If this assumption produces a contradiction, then the original query is proved.

Step 4: Resolution

From Rule 3:

$\neg \text{ViralInfection}(\text{Ravi}) \vee \text{Rest}(\text{Ravi})$

and the negated query:

$\neg \text{Rest}(\text{Ravi})$

Resolution gives:

$\neg \text{ViralInfection}(\text{Ravi})$

Step 5: Resolve with Rule 2

Rule 2:

$\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$

We already have:

$\neg \text{ViralInfection}(\text{Ravi})$

Resolving them gives:

$\neg \text{Flu}(\text{Ravi}) \vee \neg \text{BodyPain}(\text{Ravi})$

Since $\text{BodyPain}(\text{Ravi})$ is a known fact:

$\neg \text{Flu}(\text{Ravi})$

Step 6: Resolve with Rule 1

Rule 1:

$\neg \text{Fever}(\text{Ravi}) \vee \neg \text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$

We have:

$\neg \text{Flu}(\text{Ravi})$

Resolving gives:

$\neg \text{Fever}(\text{Ravi}) \vee \neg \text{Cough}(\text{Ravi})$

Since both are known facts:

$\text{Fever}(\text{Ravi})$

and

$\text{Cough}(\text{Ravi})$

we obtain the **empty clause**:

□

The empty clause represents a **contradiction**.

Resolution Proof

Knowledge Base + $\neg \text{Rest}(\text{Ravi})$

|



$\neg \text{ViralInfection}(\text{Ravi})$

|



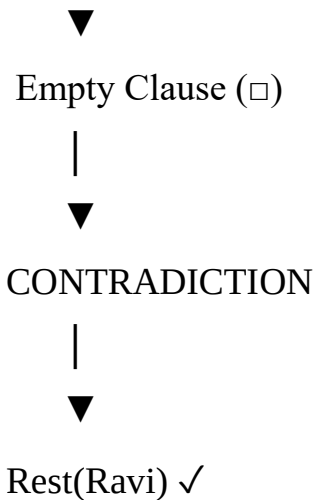
$\neg \text{Flu}(\text{Ravi})$

|



$\neg \text{Fever}(\text{Ravi}) \vee \neg \text{Cough}(\text{Ravi})$

|



Final Conclusion

Since assuming:

$\neg \text{Rest(Ravi)}$

leads to a contradiction (empty clause), the negated query is false.

Therefore:

$\text{KB} \models \text{Rest(Ravi)}$

Hence, by resolution, it is proved that Ravi should be advised to rest.

8. Explain the Final Inference Produced by the System

The system starts with the known facts about Ravi:

Fever(Ravi) Cough(Ravi) BodyPain(Ravi)

The inference engine applies the given rules step-by-step.

Step 1: Infer Flu

The first rule states:

$$\text{Fever}(x) \wedge \text{Cough}(x) \rightarrow \text{Flu}(x)$$

Since Ravi has both fever and cough:

$$\text{Fever}(\text{Ravi}) \wedge \text{Cough}(\text{Ravi})$$

the system infers:

$$\text{Flu}(\text{Ravi})$$

Therefore, **Ravi may have flu.**

Step 2: Infer Viral Infection

The second rule states:

$$\text{Flu}(x) \wedge \text{BodyPain}(x) \rightarrow \text{ViralInfection}(x)$$

We already know:

$$\text{Flu}(\text{Ravi})$$

and:

$$\text{BodyPain}(\text{Ravi})$$

Therefore, the system infers:

ViralInfection(Ravi)

Thus, **Ravi may have a viral infection.**

Step 3: Infer Need for Rest

The third rule states:

$\text{ViralInfection}(x) \rightarrow \text{Rest}(x)$

Since:

$\text{ViralInfection}(\text{Ravi})$

the system concludes:

$\text{Rest}(\text{Ravi})$

Therefore, **Ravi should be advised to rest.**

Final Inference Chain

$\text{Fever}(\text{Ravi}) + \text{Cough}(\text{Ravi})$

↓

$\text{Flu}(\text{Ravi})$

↓

$\text{Flu}(\text{Ravi}) + \text{BodyPain}(\text{Ravi})$

↓

$\text{ViralInfection}(\text{Ravi})$

↓

Rest(Ravi)

Final Conclusion

The AI knowledge-based system successfully combines the initial symptoms with the rules in the knowledge base and produces the final inference:

Ravi may have a viral infection and should be advised to rest.

The inference is logically derived from the **given knowledge base** and should not be treated as an actual medical diagnosis.

PROTOTYPE IMPLEMENTATION:

```
# =====  
  
# SMART HEALTHCARE DIAGNOSIS SYSTEM  
  
# Artificial Intelligence and Expert Systems - MLA01  
  
# =====  
  
  
# =====  
  
# SMART HEALTHCARE DIAGNOSIS SYSTEM  
  
# Prototype Implementation  
  
# =====  
  
  
print("=" * 60)  
  
print("    SMART HEALTHCARE DIAGNOSIS SYSTEM")  
  
print("=" * 60)  
  
  
# -----
```

```
# PATIENT INPUT
```

```
# -----
```

```
patient = input("\nEnter patient name: ")
```

```
print("\nEnter patient's symptoms:")
```

```
fever = input("Does the patient have fever? (yes/no): ").lower()
```

```
cough = input("Does the patient have cough? (yes/no): ").lower()
```

```
body_pain = input("Does the patient have body pain? (yes/no): ").lower()
```

```
# -----
```

```
# KNOWLEDGE BASE
```

```
# -----
```

```
facts = set()
```

```
if fever == "yes":
```

```
    facts.add("Fever")
```

```
if cough == "yes":
```

```
    facts.add("Cough")
```

```
if body_pain == "yes":
```

```
facts.add("BodyPain")
```

```
print("\n" + "-" * 60)
```

```
print("KNOWLEDGE BASE")
```

```
print("-" * 60)
```

```
print("Facts stored:")
```

```
for fact in facts:
```

```
    print(" ", fact)
```

```
# -----
```

```
# INFERENCE ENGINE
```

```
# FORWARD CHAINING
```

```
# -----
```

```
print("\n" + "-" * 60)
```

```
print("INFERENCE PROCESS")
```

```
print("-" * 60)
```

```
# Rule 1
```

```
if "Fever" in facts and "Cough" in facts:
```

```
facts.add("Flu")
```

```
print("\nRule 1:")
```

```
print("Fever AND Cough -> Flu")
```

```
print("Inference: Flu(Ravi) derived")
```

```
# Rule 2
```

```
if "Flu" in facts and "BodyPain" in facts:
```

```
facts.add("ViralInfection")
```

```
print("\nRule 2:")
```

```
print("Flu AND BodyPain -> Viral Infection")
```

```
print("Inference: ViralInfection(Ravi) derived")
```

```
# Rule 3
```

```
if "ViralInfection" in facts:
```

```
facts.add("Rest")
```

```
print("\nRule 3:")
```

```
print("Viral Infection -> Rest")
```



```
print("Inference: Rest(Ravi) derived")
```

```
# -----
```

```
# FINAL RESULT
```

```
# -----
```

```
print("\n" + "=" * 60)
```

```
print("          FINAL RESULT")
```

```
print("=" * 60)
```

```
print("\nPatient:", patient)
```

```
print("\nSymptoms:")
```

```
if "Fever" in facts:
```

```
    print(" ✓ Fever")
```

```
if "Cough" in facts:
```

```
    print(" ✓ Cough")
```

```
if "BodyPain" in facts:
```

```
    print(" ✓ Body Pain")
```

```
print("\nDerived Conditions:")
```

```
if "Flu" in facts:
```

```
    print(" ✓ Possible Flu")
```

```
if "ViralInfection" in facts:
```

```
    print(" ✓ Possible Viral Infection")
```

```
if "Rest" in facts:
```

```
    print(" ✓ Rest Recommended")
```

```
# -----
```

```
# FINAL INFERENCE
```

```
# -----
```

```
print("\n" + "-" * 60)
```

```
print("FINAL INFERENCE")
```

```
print("-" * 60)
```

```
if "Rest" in facts:
```

```
    print("Ravi may have a viral infection.")
```

```
    print("Ravi should be advised to REST.")
```

else:

```
print("The available facts are insufficient to derive the conclusion.")
```

```
print("\nNote: This is an AI knowledge representation prototype")
```

```
print("and not a real medical diagnosis.")
```

OUTPUT

```
PS C:\Users\user\Downloads\Smart_Healthcare_AI> py health_care_prototype.py
SMART HEALTHCARE DIAGNOSIS SYSTEM
=====

Enter patient name: ravi

Enter patient's symptoms:
Does the patient have fever? (yes/no): yes
Does the patient have cough? (yes/no): yes
Does the patient have body pain? (yes/no): yes

-----
KNOWLEDGE BASE
-----
Facts stored:
    Fever
    Cough
    BodyPain

-----
INFERENCE PROCESS
-----

Rule 1:
Fever AND Cough -> Flu
Inference: Flu(Ravi) derived

Rule 2:
Flu AND BodyPain -> Viral Infection
Inference: ViralInfection(Ravi) derived
```

```
Rule 3:
Viral Infection -> Rest
Inference: Rest(Ravi) derived

=====
FINAL RESULT
=====

Patient: ravi

Symptoms:
    ✓ Fever
    ✓ Cough
    ✓ Body Pain

Derived Conditions:
    ✓ Possible Flu
    ✓ Possible Viral Infection
    ✓ Rest Recommended

-----
FINAL INFERENCE
-----

Ravi may have a viral infection.
Ravi should be advised to REST.

Note: This is an AI knowledge representation prototype
and not a real medical diagnosis.
PS C:\Users\user\Downloads\Smart_Healthcare_AI>
```